

PROJECT CHARTER

Neighborly – Hyperlocal Borrowing Platform

Project Name	Neighborly – Hyperlocal Borrowing Platform
Project Team	Team C: Lea-Marie, Manar, Aysenur, Nicole, Jessica
Course	Digital Literacy IV: Software Architecture Summer Semester 2026
Instructor	Pallabi Mukherjee
Institution	Hochschule für Wirtschaft und Recht Berlin
Date	June 2026

Digital Literacy IV: Software Architecture
Hochschule für Wirtschaft und Recht Berlin | Summer Semester 2026

1. Executive Summary

Neighborly is a hyperlocal borrowing platform designed to connect urban residents who own seldom-used household items with neighbors who need those items for short-term tasks. The platform enables residents within a defined geographic area to list items they own, search for items available nearby, request to borrow them, and coordinate the exchange through an integrated in-app chat system.

The core motivation is to reduce unnecessary consumption and cost. When a neighbor needs a drill for a single task, the current options, buying, renting from a shop, or informally asking around, are either wasteful, expensive, or inconvenient. Neighborly removes this friction by providing a trusted, easy-to-use digital space where neighbors can help each other.

This document defines the project scope, functional capabilities, non-functional quality requirements, key stakeholders, the chosen architecture pattern including a trade-off analysis, and the C4 system visualizations (Context and Container level) that will guide the system design and implementation.

2. Problem Statement

2.1 The Problem

Many urban residents own tools, appliances, and equipment that they use only occasionally: a power drill, a pasta maker, a camping tent, a projector. At the same time, their neighbors frequently need exactly those items for a single task or short period, but face a set of costly and inconvenient options:

- **Buying new:** wasteful and expensive for a one-time need
- **Renting from a shop:** high cost, travel required, complex rental process
- **Informal asking:** reliant on knowing the right person, no discovery mechanism, socially awkward

The result is unnecessary spending, environmental waste from duplicate purchases, and missed opportunities for community connection. The problem is compounded by the absence of a trusted, low-friction digital platform specifically designed for this purpose at the neighborhood level.

2.2 The Opportunity

There is a clear opportunity to build a platform that leverages the density of urban neighborhoods. In a typical apartment building or city block, dozens of households collectively own hundreds of useful items. A platform that makes those items discoverable and borrowable within walking distance creates significant value for residents, saving money, reducing consumption, and strengthening community trust.

2.3 The Solution Concept: Neighborly

Neighborly is a web-based, mobile-friendly platform where residents can:

- List items they own and are willing to lend, with photos, descriptions, and availability status
- Search for available items using geolocation-scoped queries (keyword search filtered by proximity radius)
- Send and manage borrow requests directly to item owners

- Communicate with lenders or borrowers via an integrated, human-to-human in-app chat system
- Build trust through a bilateral ratings and reviews system after each completed transaction
- Protect lenders through a return-confirmation workflow with condition photos: a transaction only counts as completed once the lender confirms the item came back in good condition
- Optionally charge a small rental fee and/or require a security deposit, handled as a card pre-authorisation hold through an integrated payment provider and released automatically on confirmed return

The platform is explicitly neighborhood-scoped: search results are bounded by geographic proximity, ensuring that borrowing remains practical and that users interact with people in their actual community.

3. Stakeholders

The following stakeholders have been identified for the Neighborly platform. Each stakeholder group has distinct needs that the system must address, and these needs directly inform the functional and non-functional requirements defined in Sections 4 and 5.

Stakeholder	Role	Primary Concern
Borrower	Searches for and requests items	Fast, geo-accurate search results; easy chat to coordinate with lender; trust that items are well-described
Lender (Item Owner)	Lists items and manages requests	Easy listing creation; control over who borrows their items; assurance that lent items come back in good condition (return confirmation, photos, deposit); optional compensation via rental fees; reliable rating system to protect reputation
Both (Borrower & Lender)	Engage in borrow transaction	Safe, private, and low-friction exchange; reliable messaging; fair ratings
Platform Administrator	Manages platform health	Content moderation tools; ability to remove fraudulent listings or users; system uptime and reliability

4. Functional Requirements

Functional requirements describe what the system does and includes the specific capabilities and behaviors that Neighborly must provide to its users. Each requirement is assigned a unique identifier for traceability into the Software Design Document and the implementation.

ID	Feature	Description
FR-01	User Registration & Login	Users can register with an email and password and log in securely. Each user has a profile with a display name, photo, and general neighborhood area (no precise address required).

ID	Feature	Description
FR-02	Item Listing	Authenticated users can create a listing for an item they own. A listing includes: item name, category, description, photo, and availability status (available / not available).
FR-03	Geolocation-Based Search	Users can search for items by keyword and filter results by proximity. The system returns only items listed within a configurable radius of the user's location. Users can adjust the search radius.
FR-04	Item Detail View	Users can view a full detail page for any item, including photos, description, availability status, and the lender's profile and rating.
FR-05	Borrow Request	A borrower can send a borrow request to an item owner, specifying the intended borrow dates. The owner receives a notification of the request.
FR-06	Request Management	Item owners can accept or decline incoming borrow requests. Accepted requests update the item's availability status to 'unavailable' for the requested period.
FR-07	In-App Messaging (Chat)	When a borrow request is sent, a dedicated chat thread is opened between the borrower and the lender. Both parties can exchange messages in real time to coordinate pickup, condition, and return. Communication is human-to-human.
FR-08	Ratings & Reviews	After a borrowing transaction is completed (i.e., the lender has confirmed the item's return, see FR-11), both the borrower and lender can rate and review each other (1–5 stars with an optional text comment). Ratings are displayed on each user's profile.
FR-09	My Listings Dashboard	Owners can view, edit, and delete their own listings and manage the availability status of each item from a personal dashboard.
FR-10	Notifications	Users receive in-app notifications for: new borrow requests, request acceptance/decline, new messages, return confirmations, deposit holds and releases, and completed transaction review prompts.
FR-11	Return Confirmation & Lender Protection	Every accepted borrow request follows a defined status lifecycle: accepted → handed over → return reported → completed. The borrower reports the item as returned; the transaction is only marked 'completed' once the lender confirms that the item came back in good condition. This confirmation unlocks the ratings step (FR-08) and automatically releases the deposit hold (FR-13). If the lender instead reports a problem, the request is set to 'disputed', is visibly flagged on the borrower's profile until resolved, and triggers the deposit capture defined in FR-13.
FR-12	Condition Photo Verification	At hand-over, the lender uploads photos documenting the item's condition; before the lender can confirm the return (FR-11), the borrower must upload photos of the returned item. All photos are timestamped and attached to the borrow request, creating an evidence trail that protects both parties in case of a dispute.

ID	Feature	Description
FR-13	Payments & Deposits	A lender can optionally set a rental fee and/or a security deposit on a listing. When a borrow request is accepted, the borrower pays the fee and the deposit amount is placed as a pre-authorisation hold via an integrated payment provider. On confirmed return the hold is released automatically; on a dispute the deposit is captured until the matter is resolved. Card data is tokenised by the provider's client-side SDK and never touches Neighborly's servers.

5. Non-Functional Requirements

Non-functional requirements (NFRs) describe how the system performs, and they are stated in a measurable and testable form. These requirements correspond directly to the quality attributes covered in this course: performance, usability, reliability, security, modifiability, availability, and scalability. Each NFR is connected to a specific stakeholder need and architectural concern.

ID	Attribute	Measurable Requirement	Rationale
NFR-01	Performance	Geolocation search results must be returned and displayed within 2 seconds under normal load (up to 100 concurrent users).	Slow search destroys the 'low-friction' promise of the platform.
NFR-02	Usability	A new user must be able to complete registration and publish their first item listing within 3 minutes, without any external guidance.	The platform's success depends on fast onboarding for non-technical neighbors.
NFR-03	Reliability	Chat messages must be delivered to the recipient with no message loss. In the event of a temporary connection failure, messages must be queued and delivered once connection is restored.	Trust between neighbors depends on reliable communication; a missed message could break a borrowing agreement.
NFR-04	Security	User precise location (GPS coordinates) must never be exposed to other users. Only the neighborhood-scoped search radius is used for matching; the item listing shows only a general area (e.g., 'Prenzlauer Berg').	Privacy is critical for user adoption. Exposing home addresses would deter users from joining the platform.
NFR-05	Security	All user authentication must use hashed passwords (bcrypt or equivalent). Sessions must expire after 7 days of inactivity.	Prevents unauthorized account access and protects the trust layer.
NFR-06	Modifiability	New item categories (e.g., 'Camping Gear') must be addable through a configuration change only, with no modification required to the search or listing logic.	The item taxonomy will grow over time; tight coupling to categories would make expansion costly.
NFR-07	Availability	The platform must maintain at least 99% uptime during peak hours (weekday evenings 17:00–22:00 CET).	Neighbors are most active in the evenings; downtime

ID	Attribute	Measurable Requirement	Rationale
			during this window has the highest impact on usability.
NFR-08	Scalability	The system architecture must support horizontal scaling of the search and listing functionality to handle growth beyond the initial neighborhood scope without redesigning core components.	If the concept succeeds, it must scale to additional neighborhoods without an architectural overhaul.
NFR-09	Security	All card data must be handled exclusively by the payment provider's client-side SDK (tokenisation). Neighborly's servers must never store, log, or transmit raw card numbers.	Delegating PCI-DSS compliance to the provider keeps payment risk and regulatory burden off the platform.

5.1 Quality Attribute Scenarios

To make the most critical NFRs concrete and testable, the three highest-risk quality attributes are specified below as six-part quality attribute scenarios (source of stimulus, stimulus, environment, artifact, response, response measure). This format forces each requirement to be unambiguous: it states exactly who triggers the system, under which conditions, and how we will measure whether the architecture meets the requirement.

Scenario 1: Performance (refines NFR-01)

Source of stimulus	A registered resident using the platform on a mobile browser
Stimulus	Submits a keyword search (e.g., 'drill') with a 1 km radius filter
Environment	Normal operation, up to 100 concurrent users
Artifact	Search module (business logic layer) and PostgreSQL database (data layer)
Response	The system executes an indexed geo-query and returns all matching listings within the radius
Response measure	Results are rendered in the client within 2 seconds in at least 95% of requests

Scenario 2: Reliability (refines NFR-03)

Source of stimulus	A borrower in an active chat thread with a lender
Stimulus	The borrower's connection drops while a message is being sent
Environment	Degraded network conditions (mobile connection loss)
Artifact	Real-time messaging container (Socket.io) and message persistence in the database
Response	The message is persisted server-side and queued; delivery is retried automatically once the recipient reconnects
Response measure	Zero lost messages; queued messages are delivered within 5 seconds of reconnection

Scenario 3 — Security (refines NFR-04)

Source of stimulus	A curious or malicious registered user
Stimulus	Attempts to obtain another user's exact GPS coordinates through the API (e.g., by inspecting search responses)
Environment	Normal operation
Artifact	API application (presentation and business logic layers)
Response	API responses contain only the general neighborhood label; latitude/longitude values are used server-side for radius filtering and are never serialized into any client-facing response
Response measure	An audit of all API endpoints confirms zero endpoints exposing another user's coordinates

6. Execution Plan and Architectural Approach

This section addresses the key architectural and implementation decisions for the two most critical subsystems in Neighborly, the search system and the chat/messaging system, and documents the chosen architecture pattern together with the trade-offs of that decision.

6.1 Search System: Geolocation-Based Search

The search system will be geolocation-based, combining keyword matching with proximity filtering. The approach works as follows:

- When a user searches, they provide a keyword (e.g., 'drill') and the system uses their current GPS coordinates (obtained via the browser Geolocation API with explicit permission).
- Item listings stored in the database include the latitude/longitude of the lender's location, captured when the listing is created.
- The search query filters results by keyword AND by geographic radius. The default radius is 1 km, and users can adjust it (e.g., 500 m to 5 km).
- Users search by item, not by location. The location is a filter, not the primary search parameter.

Architectural implications: Search is implemented as a dedicated module within the business logic layer, backed by indexed location and category fields in the database. This supports NFR-01 (performance) (geo-queries run against indexes rather than full table scans) and avoids N+1 query patterns by resolving listings and lender ratings in a single joined query.

6.2 Chat System: Human-to-Human Messaging

The messaging system will be fully human-to-human. No chatbot or automated assistant will be involved. The design rationale is:

- Borrowing a neighbor's item is a personal, trust-based interaction. A chatbot would feel impersonal and undermine the community-first mission of the platform.
- The coordination required (pickup time, item condition, return date) is context-specific and requires natural language conversation between two people.

- A human chat model is significantly simpler to implement reliably, which reduces development risk.

Implementation approach: When a borrower submits a borrow request, the system automatically creates a chat thread between the borrower and the lender. Both parties can send text messages within that thread, and the thread persists until the transaction is closed.

Architectural implications: Real-time messaging runs as a separate container, decoupled from the request/response API. This satisfies NFR-03 (reliability) by isolating message delivery and queueing logic, and NFR-04 (security) by scoping each chat thread strictly to the two parties of a transaction.

6.3 Chosen Architecture Pattern: Layered Client-Server

Neighborly follows a client-server architecture with a layered backend. The client is a React single-page application (presentation on the client side); the server is a single deployable Node.js/Express application organized into three layers:

- **Presentation layer (routes):** Express route definitions that receive HTTP requests, validate input, and translate between JSON and internal data structures. This layer contains no business rules.
- **Business logic layer (controllers and services):** all domain rules live here, who may borrow what, when an item becomes unavailable, which status transitions a borrow request may take (FR-11), how a rating is calculated. The layer is internally organized into cohesive modules: User/Auth, Listing, Search, Messaging, Rating & Review, Payment, and Notification.
- **Data layer (models):** database models and queries against PostgreSQL. Only this layer talks to the database, so a change in the persistence technology would be contained here.

An important clarification: the six modules listed above are modules inside the business logic layer of one deployable backend, they are not independently deployed microservices. Each module exposes a clear interface to the other modules and hides its internal implementation. This distinction matters for the trade-off analysis in Section 6.4.

The structure deliberately applies the core design principles from this course:

- **Separation of concerns:** each layer has exactly one responsibility (translate, decide, persist), so layers can be developed and tested independently.
- **High cohesion, low coupling:** each module groups closely related functionality (e.g., everything about ratings lives in one module) and communicates with other modules only through defined interfaces.
- **Encapsulation:** modules hide their internals; for example, no module other than Search knows how the radius query is executed.
- **Deferred binding:** item categories are configuration data rather than hardcoded values (NFR-06), and the module boundaries allow a later switch from REST to event-driven messaging without rewriting business logic.

6.4 Architecture Trade-Off Analysis

Choosing an architecture means accepting its weaknesses along with its strengths. Three realistic candidate patterns were compared before deciding:

Candidate Pattern	Strengths for Neighborly	Decision and Reasoning
Layered Client-Server (chosen)	Simple to build, test, and demo; one deployment unit; clear visible structure (routes → controllers/services → models); fits a 7-person student team and a ~5-minute demo.	Chosen. The single server is a potential bottleneck, but at our scale (one neighborhood, ≤100 concurrent users) this risk is acceptable and can be mitigated later with a load balancer and caching, because the API is stateless.
Microservices	Independent deployment and scaling per service; failure isolation; technology freedom per service.	Rejected. The operational overhead (service discovery, distributed data, inter-service communication, monitoring) is disproportionate for an MVP of this size. We deliberately sacrifice independent scaling per service, but our module boundaries mean individual modules could be extracted into services later if load demands it.
Event-Driven Architecture	Naturally fits notifications and chat; high decoupling between producers and consumers; good elasticity.	Rejected as the overall pattern. Asynchronous flows are harder to reason about and debug, and most of our use cases (search, listing, requests) are synchronous request/response. We adopt the idea selectively: chat delivery is event-based via WebSockets, inside an otherwise layered system.

What we gained: a system the whole team can understand, an architecture whose layers are directly visible in the folder structure of the codebase, and low operational complexity.

What we sacrificed: per-component scaling and deployment independence. **Why this is acceptable:** NFR-08 demands the ability to scale, not microservices. A stateless layered backend scales horizontally by running multiple identical instances behind a load balancer, which satisfies NFR-08 without redesign.

7. System Visualization (C4 Model)

Following the C4 model (Brown, 2018), the system is documented at two levels of abstraction: the System Context diagram (Level 1) shows Neighborly as a black box in its environment, and the Container diagram (Level 2) opens that box to show the deployable building blocks and the technologies inside it.

7.1 System Context Diagram (Level 1)

The context diagram answers the question: who uses the system and what does it depend on? Three personas interact with Neighborly: borrowers, lenders (in practice often the same residents in different roles), and the platform administrator. The system depends on three external systems: the browser Geolocation API for the user's approximate position (with explicit consent, supporting NFR-04), a cloud file storage service for item and condition photos, and a payment provider that handles rental fees and deposit holds (FR-13, NFR-09).

System Context Diagram [C4 - Level 1]

Neighborly - Hyperlocal Borrowing Platform

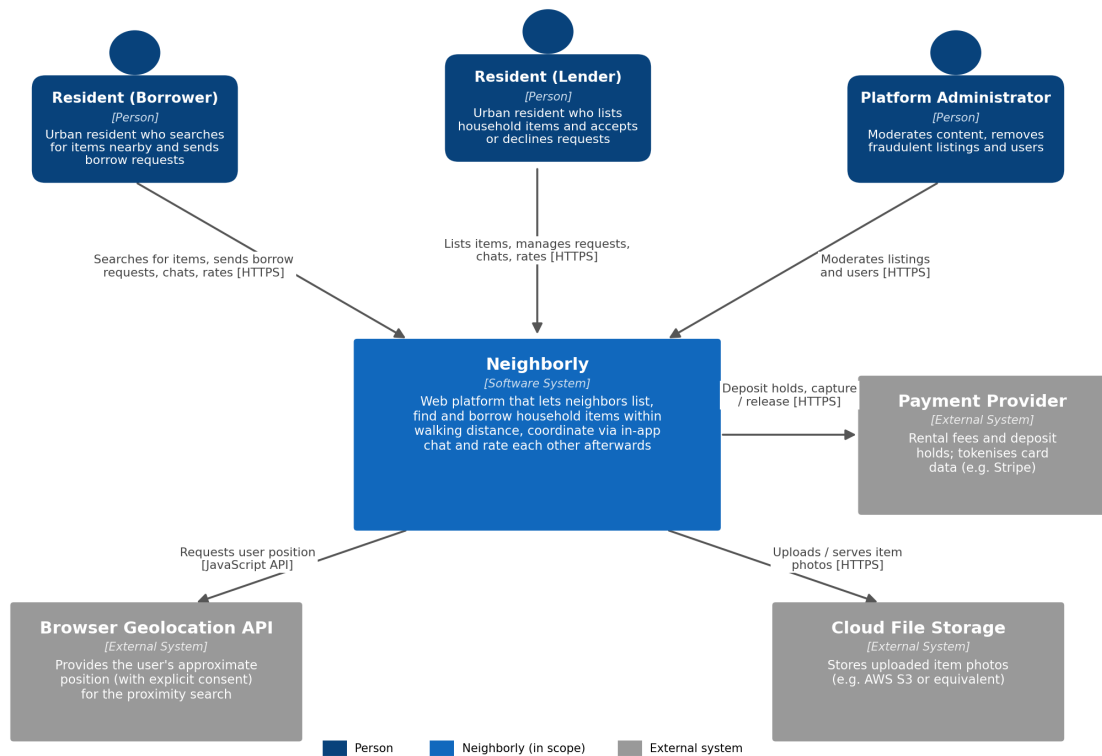


Figure 1: System Context Diagram (C4 Level 1) for Neighborly

7.2 Container Diagram (Level 2)

The container diagram shows the four building blocks of the system: the React single-page application (presentation), the Express API application (whose internal layering, routes, controllers/services, models, implements the pattern described in Section 6.3), the Socket.io real-time messaging container (isolated for reliability, NFR-03), and the PostgreSQL database with indexed location and category fields (performance, NFR-01). Item photos are stored externally and uploaded via pre-signed URLs so that large files never pass through the API server. Payment operations (deposit holds, capture, release) are delegated to the external payment provider via its server-side API, while card details are tokenised directly in the browser by the provider's SDK, they never reach a Neighborly container (NFR-09).

Container Diagram [C4 - Level 2]

Neighborly - Hyperlocal Borrowing Platform

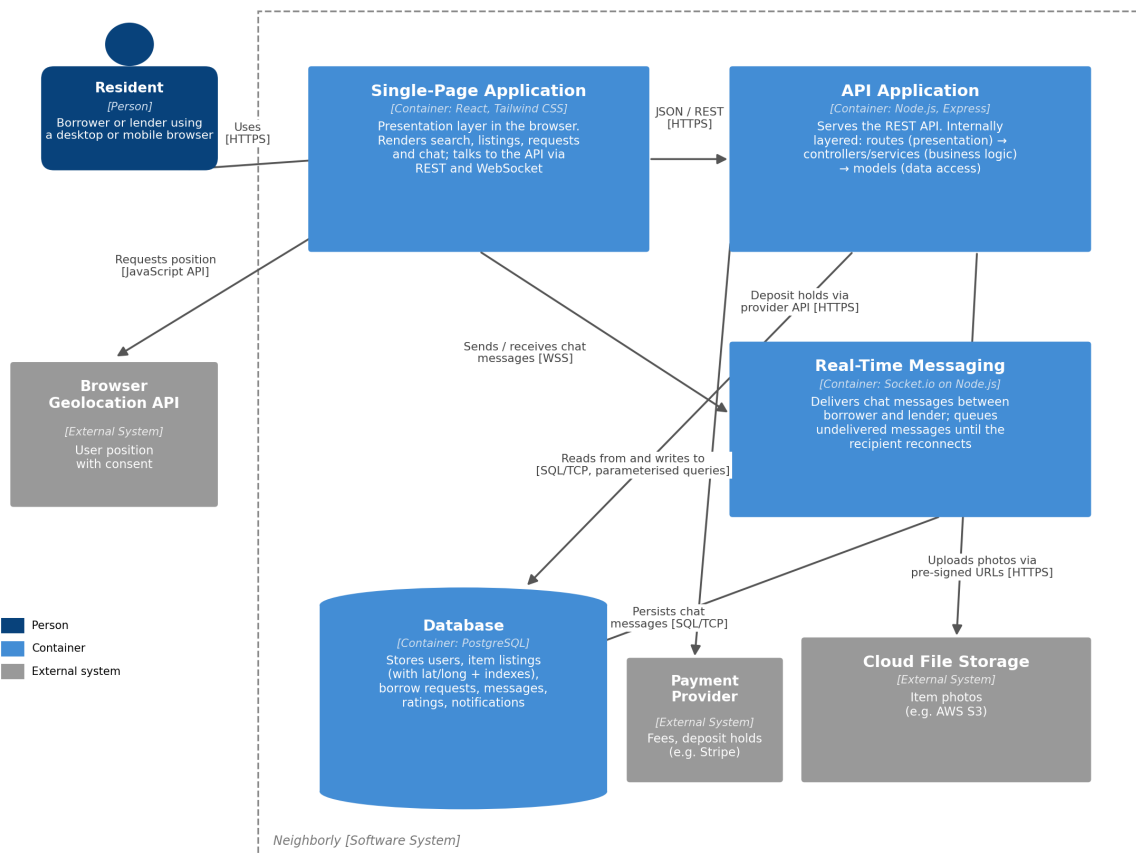


Figure 2: Container Diagram (C4 Level 2) for Neighborly

8. Scope and Constraints

8.1 In Scope

- Web-based application (mobile-responsive)
- User registration, authentication, and profile management
- Item listing creation, editing, and deletion
- Geolocation-based item search with configurable radius
- Borrow request workflow with full status lifecycle (request, accept/decline, hand-over with condition photos, return report with photos, lender confirmation)

- Optional rental fees and security-deposit holds via an integrated payment provider, using the provider's free sandbox/test environment (no real money is moved in the course demo)
- In-app human-to-human messaging (chat threads per request)
- Ratings and reviews system
- In-app notifications

8.2 Out of Scope (for this project)

- Live (real-money) payment operation: the MVP integrates the payment provider in sandbox mode only. Production payouts, a platform commission, tax handling (e.g., VAT on rental fees), and KYC/identity verification of lenders are deliberately deferred until after the course.
- Manual dispute arbitration by the platform: the deposit capture (FR-13) follows the lender's report; a human moderation process for contested captures is future work.
- Native mobile applications (iOS / Android); the web application will be mobile-responsive
- AI-powered chatbot or recommendation engine
- Multi-language support (English only for the MVP)
- Integration with external mapping or logistics providers

8.3 Assumptions

- Users are assumed to be urban residents with smartphone or desktop browser access.
- Users consent to sharing their approximate location for the search radius functionality.
- The platform will be deployed as a cloud-hosted web application (running locally with a demo script is sufficient for the course submission).
- Item photos will be stored using a cloud file storage service (e.g., AWS S3 or equivalent).
- The payment provider's free sandbox environment (e.g., Stripe test mode) is sufficient to demonstrate fee and deposit flows end-to-end; no real money is moved during the course project.

9. Project Deliverables

The following deliverables will be produced throughout the project lifecycle, in alignment with the course requirements:

- **Project Charter (this document)** – defines scope, requirements, architecture pattern, trade-offs, and execution plan
- **Software Design Document (SDD)** – detailed component design, design pattern decisions, and refinement of the C4 diagrams introduced in Section 7
- **Working Application** – a functional implementation of the core features (item listing, geolocation search, borrow request, in-app chat), reflecting the chosen layered architecture visibly in the code structure
- **Demo Video** – a recorded walkthrough demonstrating all implemented features clearly

- **Final Group Presentation** – live presentation covering architecture clarity, concept understanding, analysis and reasoning, and project storytelling

10. Quality Attribute Summary

The table below summarizes how the most important quality attributes manifest in Neighborly, and how they shape the architectural decisions in this document and the upcoming SDD. This connects stakeholder needs to architecture, following the principle that quality attributes drive architectural structure.

Quality Attribute	How It Manifests in Neighborly	Architectural Implication
Performance	Search must respond within 2 seconds	Indexed geo-queries in the data layer; dedicated search module; no N+1 query patterns
Usability	New users must onboard in under 3 minutes	Simple UI flows; minimal required fields for listing creation
Reliability	Messages must not be lost during connection drops	Message queuing in the messaging container; persistent storage before delivery confirmation
Security	Precise location must never be exposed; passwords must be hashed; card data never on our servers	Coordinate abstraction in API responses; bcrypt password hashing; JWT sessions; HTTPS; card tokenisation delegated to the payment provider
Modifiability	New item categories added without code changes	Categories managed as configuration, not a hardcoded enum; layered structure isolates change
Availability	99% uptime during evening peak hours	Cloud hosting with health monitoring and auto-restart; stateless API allows redundant instances
Scalability	System must scale beyond the initial neighborhood without redesign	Stateless layered backend scales horizontally behind a load balancer; modules extractable into services if needed

11. References

Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley.

Brown, S. (2018). The C4 Model for Visualising Software Architecture. Retrieved from <https://c4model.com>

Richards, M., & Ford, N. (2020). Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media.